

Delphi

A Multi-Agent World-Simulation Engine
Extended for Financial-Market Scenario Prediction — Documentation

Vito Ciciretti

July 2026

Contents

1 Overview	1
1.1 Intended use	1
2 Provenance	2
3 Workflow	2
4 Scenario / Domain Presets	2
4.1 What a preset controls	3
4.2 Built-in presets	4
4.3 Selecting and adding scenarios	4
5 Setup	5
5.1 Prerequisites	5
5.2 Source deployment	5
5.3 Docker deployment	5
6 License & Attribution	5

1 Overview

Delphi builds a high-fidelity parallel digital world from real-world seed material — a news article, a policy draft, a financial signal, or a story to re-imagine. Inside that world, agents with independent personalities, long-term memory, and behavioural logic interact and evolve. Variables can be injected from a god’s-eye view to watch how the resulting dynamics play out — rehearsing outcomes in a sandbox before they happen for real.

Given seed material and a natural-language prediction question, Delphi returns a detailed report and an interactive digital world that remains explorable after the simulation ends.

1.1 Intended use

- **Macro** — a zero-risk rehearsal lab for decision-makers: test policies, messaging, and market moves before committing to them.
- **Micro** — a creative sandbox: deduce alternate story endings or explore “what if” scenarios.

2 Provenance

Delphi is a **derivative work**. Its simulation core is [MiroFish](#), an AGPL-3.0 multi-agent prediction engine, and its underlying agent-interaction substrate is [OASIS](#) (Open Agent Social Interaction Simulations) from the CAMEL-AI team. Delphi preserves the AGPL-3.0 license end to end, as required. The full list of modifications is tracked in `NOTICE.md` in the repository root.

What is original to Delphi is described in Section 4: a config-driven scenario/domain preset layer that generalises the engine from a single fixed scope (social-media public opinion) to arbitrary domains, including financial markets.

3 Workflow

1. **Graph building** — seed extraction, individual/collective memory injection, GraphRAG construction.
2. **Environment setup** — entity relationship extraction, persona generation, agent configuration injection.
3. **Simulation** — parallel multi-channel simulation, auto-parsed prediction requirements, dynamic temporal memory updates.
4. **Report generation** — a report agent with a tool set for deep interaction with the post-simulation world.
5. **Deep interaction** — chat with any agent in the simulated world, or with the report agent itself.

4 Scenario / Domain Presets

The upstream engine hard-wired its assumptions (rhythm, channels, stance vocabulary, prompt framing) to a single scope: social-media public-opinion simulation, on a fixed China-timezone activity rhythm. Delphi externalises all of that into **scenario presets** — config-driven JSON descriptions of what kind of world a simulation runs in — so the same engine can model social media, financial markets, organisational decisions, or narrative worlds without touching engine code.

4.1 What a preset controls

Field	Meaning
<code>id</code>	Unique preset identifier (used in the API / <code>SCENARIO_DEFAULT</code>).
<code>name / description</code>	Human-facing label and blurb shown in the UI.
<code>domain</code>	High-level grouping: <code>social_media</code> , <code>market</code> , <code>organization</code> , <code>narrative</code> , <code>custom</code> .
<code>activity_rhythm</code>	When agents are active over 24h (peak / work / morning / off-peak hours and multipliers) plus a time-zone label. Replaces the old fixed China-timezone rhythm.
<code>default_total_hours / default_minutes_per_round</code>	Default simulation horizon and time granularity.
<code>channels</code>	Communication channels agents use; each maps onto an <code>engine_platform</code> (<code>twitter</code> or <code>reddit</code> for OASIS) with recommender weights, a viral threshold, and echo-chamber strength.
<code>stances / default_stance</code>	Domain-appropriate stance vocabulary (e.g. bullish/bearish for markets, advocate/skeptic for organisations).
<code>prompt_framing</code>	Natural-language framing injected into LLM prompts so generated events, personas, and behaviours fit the domain.
<code>tags</code>	Free-form labels for filtering and display.

Table 1: Fields exposed by a scenario preset.

Engine note. The underlying simulation still runs on OASIS social platforms, so every channel maps onto `twitter` or `reddit`. A preset changes the framing, rhythm, and dynamics of a run; the `engine` field on each channel leaves room for alternative simulation backends later.

4.2 Built-in presets

id	Domain	Highlights
social_media	social_media	Default. Reproduces the original engine behaviour: China-timezone rhythm, 72h, Twitter + Reddit.
global_social_media	social_media	Flattened 24/7 rhythm for worldwide, cross-timezone events.
financial_market	market	Market-hours rhythm, 48h in 30-minute rounds, fast news feed + investor forum, bullish/bearish stances.
organization	organization	Business-hours rhythm, 5-day horizon, announcements + team discussion, advocate/skeptic/blocker stances.
creative_narrative	narrative	No diurnal cycle, week-long story-time horizon, in-character framing for alternate endings.

Table 2: Presets shipped with Delphi.

4.3 Selecting and adding scenarios

Pass `scenario_id` when creating a simulation:

```
POST /api/simulation/create
{
  "project_id": "proj_xxxx",
  "scenario_id": "financial_market"
}
```

If omitted, `Config.SCENARIO_DEFAULT` (env `SCENARIO_DEFAULT`, default `social_media`) is used. Which platforms run is inferred from the preset's channels unless `enable_twitter` / `enable_reddit` is passed explicitly. List everything available with `GET /api/simulation/scenarios`.

Adding a new domain requires no code: drop a `<id>.json` file into `SCENARIO_PRESETS_DIR` (default `backend/uploads/scenarios`), following the schema of the built-ins in `backend/app/scenarios/presets`. A user preset with the same `id` as a built-in overrides it, so a shipped scenario can be re-tuned without editing source. Presets are validated on load; malformed files raise a clear `ScenarioLoadError` at startup rather than failing silently at runtime.

Minimal example:

```
{
  "id": "product_launch",
  "name": "Product Launch Buzz",
  "domain": "social_media",
  "activity_rhythm": { "timezone": "UTC",
                      "peak_hours": [12, 13, 18, 19, 20] },
  "default_total_hours": 96,
  "channels": [
    { "id": "social", "engine_platform": "twitter",
      "viral_threshold": 8 },
    { "id": "forum", "engine_platform": "reddit" }
  ],
  "stances": ["excited", "critical", "neutral", "observer"],
  "default_stance": "neutral",
  "prompt_framing": "Simulate public reaction to a new product launch
  ..."
}
```

5 Setup

5.1 Prerequisites

Tool	Version	Description
Node.js	18+	Frontend runtime, includes npm
Python	≥ 3.11 , ≤ 3.12	Backend runtime
uv	latest	Python package manager

5.2 Source deployment

```
cp .env.example .env
# fill in LLM_API_KEY / LLM_BASE_URL / LLM_MODEL_NAME
# and ZEP_API_KEY (memory graph; free tier from app.getzep.com)

npm run setup:all      # install root + frontend + backend deps
npm run dev            # start frontend (:3000) + backend (:5001)
```

5.3 Docker deployment

```
cp .env.example .env
docker compose up -d --build
```

6 License & Attribution

Delphi is licensed under the GNU Affero General Public License v3.0 — see [LICENSE](#). As described in [Section 2](#), it is a derivative of MiroFish (AGPL-3.0) and depends on OASIS (CAMEL-AI). All original copyright and license notices are retained; trademarks, logos, and project names referenced from the upstream project remain the property of their respective owners. The complete, itemised list of modifications is in [NOTICE.md](#).